
Universidad de Guayaquil
Facultad de Ciencias Matemáticas y Físicas
Carrera de Software

Bitácora técnica

Prototipo web de BI asistido por IA

Estudiantes	Emily Dayan Robles Alvarado y Lesly Samay Velásquez Anchundia
Tema	Construcción asistida de un datamart, KPIs, insights y pronóstico
Repositorio local	/home/ceo/Proyectos/BI
Versión de la bitácora	0.2.3
Fecha de apertura	24 de agosto de 2026

Índice

1. Propósito y reglas de mantenimiento	1
2. Resumen del alcance vigente	1
3. Registro cronológico	1
3.1. 24 de agosto de 2026 - Inicio del repositorio y ambiente	1
3.1.1. Decisiones técnicas	2
3.1.2. Archivos y componentes principales	2
3.1.3. Incidentes y observaciones	3
3.2. 24 de agosto de 2026 - Formalización del Sprint 1 y manual técnico	3
3.3. 24 de agosto de 2026 - Gobierno Git, colaboradores y estabilidad DevOps	4
3.3.1. Incidentes y resolución	4
3.4. 24 de agosto de 2026 - Ambientes simultáneos y vista de entrega	5
3.4.1. Decisión operativa	6
3.4.2. Incidente de verificación documental	6
3.5. 24 de agosto de 2026 - Publicación abierta de imágenes GHCR	7
3.5.1. Incidentes y resolución	7
3.6. 24 de agosto de 2026 - Operación documentada de contenedores	7
3.7. 1 de septiembre de 2026 - Cierre documental y adopción de SDD	9
3.7.1. Decisión de gobierno	9
3.8. 2 de septiembre de 2026 - Profundización documental y material docente anónimo	9
3.8.1. Criterio de separación documental	10
3.9. 4 de septiembre de 2026 - Especificación UX y configuración LLM del Sprint 2	11
3.9.1. Decisión de diseño	11
3.10. 6 de septiembre de 2026 - Implementación inicial del Sprint 2	13
4. Matriz de verificación de la fase	14
5. Próximos pasos	14
6. Historial del documento	15

1. Propósito y reglas de mantenimiento

Este documento conserva la trazabilidad técnica del prototipo: decisiones, cambios, verificaciones, incidentes, evidencias y trabajo pendiente. Debe actualizarse al final de cada sesión significativa y antes de etiquetar una entrega.

Reglas:

- No eliminar incidentes ni resultados fallidos; agregar su resolución cuando exista.
- Registrar comandos y resultados relevantes sin copiar contraseñas, tokens o datos sensibles.
- Relacionar cada hito con un commit, pull request o etiqueta cuando GitHub esté disponible.
- Regenerar el PDF con `make logbook` y comprobar visualmente sus páginas.
- Mantener el archivo fuente `bitacora.tex` y el PDF generado bajo control de versiones.

2. Resumen del alcance vigente

El anteproyecto corregido define una prueba de concepto académica con una fuente activa SQL Server y AdventureWorks en modo de sólo lectura. El flujo futuro extraerá metadatos, solicitará propuestas estructuradas a un LLM, exigirá aprobación humana y validaciones determinísticas, ejecutará un ETL básico hacia PostgreSQL y mostrará cinco KPIs, tres visualizaciones, insights explicables y un pronóstico mensual mediante regresión lineal con MAPE y RMSE.

El anexo institucional exige módulos de seguridad, parámetros, negocio y reportes. Se acordó un RBAC mínimo con usuarios, roles, permisos y menús por rol. En la fase actual existe únicamente el límite arquitectónico; no hay autenticación ni lógica de negocio implementada.

3. Registro cronológico

3.1. 24 de agosto de 2026 - Inicio del repositorio y ambiente

Actividad	Estado	Resultado
Revisión del ante-proyecto	Completado	Se inspeccionaron las 11 páginas, incluyendo alcance, exclusiones, objetivos, metodología, cronograma y requisitos mínimos institucionales.
Arquitectura base	Completado	Se fijaron React con Vite y TypeScript, FastAPI, PostgreSQL interno/datamart y SQL Server AdventureWorks como fuente externa.
Docker-first	Completado	El host sólo requiere Docker Compose. Se definieron etapas de desarrollo, prueba y producción para frontend y backend.
Bases reproducibles	Completado	Se añadieron imágenes inicializadoras para PostgreSQL y SQL Server. AdventureWorks se restaura desde el respaldo oficial en un volumen vacío.

Actividad	Estado	Resultado
Seguridad	Completado	El paquete security y las reglas del RBAC quedaron documentados, sin modelos ni endpoints funcionales.
Observabilidad	Completado	Se añadieron endpoints <code>/health/live</code> y <code>/health/ready</code> , más una pantalla de estado en React.
DevOps	Completado	Se prepararon CI, CD, Dependabot, plantilla de pull request y publicación de imágenes en GHCR.
Guía de réplica	Completado	Se documentaron el entorno aislado desde cero y el consumo de imágenes publicadas.
GitHub remoto	Completado	Repositorio privado <code>LFXAS/prototipo-bi-ia</code> , rama <code>main</code> publicada y remoto <code>origin</code> configurado.
Verificación completa	Completado	El conjunto se reconstruyó desde volúmenes vacíos; los cuatro servicios quedaron saludables y las pruebas automatizadas finalizaron correctamente.

3.1.1. Decisiones técnicas

- El repositorio definitivo se ubica en `/home/ceo/Proyectos/BI`; no se codifican rutas absolutas en Compose, por lo que otras máquinas pueden clonarlo en cualquier ubicación.
- El Compose predeterminado crea red, bases y volúmenes aislados. Usa los puertos anfitrión 55432 y 51433 para no interferir con instancias existentes.
- Los volúmenes Docker no se exportan como imágenes. El estado inicial se reconstruye con respaldos públicos, scripts y futuras migraciones versionadas.
- `ApplicationIntent=ReadOnly` no es un control suficiente. El usuario `bi_reader` recibe lectura y denegaciones explícitas de escritura en SQL Server.
- CD publicará las imágenes propias en GitHub Container Registry con etiquetas de rama, versión y SHA.

3.1.2. Archivos y componentes principales

- `compose.yaml`: aplicación, PostgreSQL y SQL Server autocontenidos.
- `compose.release.yaml`: ejecución sin compilar, usando GHCR.
- `backend/`: API, configuración, sesión PostgreSQL, salud, Alembic y scaffolding modular.
- `frontend/`: interfaz React, proxy hacia la API, pruebas y Nginx de producción.
- `infra/`: inicialización de PostgreSQL, restauración de SQL Server y compilador LaTeX.
- `docs/replication-guide.md`: manual operativo paso a paso.
- `.github/workflows/`: integración y entrega continua.

3.1.3. Incidentes y observaciones

1. La carpeta generada inicialmente contenía un directorio `.git` vacío y no era todavía un repositorio válido. Se resolvió inicializando Git sobre `main`; el commit base es `ead11fb`.
2. El primer intento de generar `package-lock.json` falló porque npm trató de escribir en una caché de sólo lectura dentro del directorio personal. Se resolvió con la caché temporal `/tmp/bi-npm-cache`; el lockfile quedó generado sin vulnerabilidades reportadas.
3. Una verificación de Compose se lanzó desde la subcarpeta `frontend`; no encontró los archivos raíz. Se repitió desde la raíz y las configuraciones de desarrollo y publicación resultaron válidas.
4. La primera construcción del backend detectó formato pendiente y un import moderno requerido por Ruff. Se corrigieron ambos; Ruff, Mypy y dos pruebas Pytest finalizaron correctamente.
5. La primera prueba del frontend duplicó la inicialización de `jest-dom`. Se retiró la importación redundante; ESLint, Vitest y Vite finalizaron correctamente.
6. npm detectó vulnerabilidades en las versiones iniciales de Vite y Vitest. Se actualizaron a las versiones corregidas 7.3.6 y 3.2.7; el lockfile registra cero vulnerabilidades.
7. La primera salud del frontend consultó `localhost`, que BusyBox resolvió como IPv6 mientras Vite escuchaba en IPv4. Se cambió la sonda a `127.0.0.1` y el contenedor quedó saludable.
8. La primera restauración creó `bi_reader`, pero una denegación demasiado amplia de `CONTROL` también bloqueó el permiso subordinado de conexión. Se eliminó esa denegación, la sonda de salud pasó a probar el usuario real y se reconstruyó todo desde volúmenes vacíos.
9. La primera restauración de AdventureWorks requiere red y puede tardar varios minutos. Los reinicios posteriores reutilizan el volumen persistente.
10. La primera ejecución de Actions finalizó correctamente, pero advirtió que varias acciones todavía declaraban Node.js 20. Se actualizaron a las versiones oficiales vigentes: `checkout@v7`, `upload-artifact@v7` y acciones Docker de generaciones 4, 6 y 7.
11. La búsqueda autorizada de tres correos no devolvió cuentas públicas y la API de colaboradores exige nombres de usuario. Las invitaciones quedan pendientes hasta obtener los identificadores GitHub o iniciar sesión en la interfaz web.

3.2. 24 de agosto de 2026 - Formalización del Sprint 1 y manual técnico

- Se formalizó la fase de ambiente como **Sprint 1: ambiente reproducible, repositorio y DevOps**, con objetivo, backlog, criterios de aceptación, revisión, incidencias y retrospectiva.
- Se creó el informe académico `docs/sprints/sprint-01-entorno.tex` y su PDF, acompañado por capturas reales del frontend y OpenAPI.
- Se creó el manual `docs/manual-tecnico/manual-tecnico.tex` y su PDF con instalación, configuración, réplica, operación, pruebas, CI/CD, seguridad, diagnóstico y recuperación.
- `make docs` regenera los tres documentos con la imagen LaTeX del proyecto; `make verify` los incluye en la definición de terminado.
- CI compila, compara y publica los tres PDF como un único artefacto de documentación técnica.

3.3. 24 de agosto de 2026 - Gobierno Git, colaboradores y estabilidad DevOps

Actividad	Estado	Resultado
Colaboradores GitHub	Completado	Se verificaron LeslyVelasquez y EmyRoblesBerry; ambas cuentas tienen permiso de escritura en LFXAS/prototipo-bi-ia.
Ramas permanentes	Completado	<code>develop</code> quedó como rama predeterminada de integración y <code>main</code> como rama estable de publicación. El trabajo usa <code>feature/*</code> , <code>fix/*</code> , <code>docs/*</code> o <code>chore/*</code> .
Visibilidad	Completado	Por autorización expresa, el repositorio cambió de privado a público para habilitar las reglas de protección sin contratar todavía GitHub Pro. La visibilidad de los paquetes GHCR se gestiona por separado.
Protección de ramas	Completado	<code>develop</code> y <code>main</code> exigen pull request, una aprobación, conversaciones resueltas, rama actualizada y ocho controles de CI; force-push y eliminación están deshabilitados.
Flujo automático	Completado	CI valida ambas ramas y rechaza promociones a <code>main</code> cuyo origen no sea <code>develop</code> ; CD continúa limitado a <code>main</code> y etiquetas <code>v*</code> .
Dependabot	Completado	Las ramas <code>dependabot/*</code> se identificaron como temporales. npm agrupa sólo parches y cambios menores; los saltos mayores quedan fuera de las actualizaciones rutinarias.
Reinicio SQL Server	Completado	El inicializador espera que AdventureWorks esté en línea antes de configurar el lector y crear la marca de salud. Un reinicio con volumen existente terminó con cuatro servicios saludables y 71 tablas.
Manual y guía	Completado	README, guía de réplica, flujo Git y manual técnico describen comandos, promociones, protecciones, Dependabot y recuperación.
Verificación final	Completado	<code>make verify</code> validó Compose, política de ramas, backend, frontend y los tres PDF; Actionlint 1.7.12 validó los flujos, todo dentro de Docker.

3.3.1. Incidentes y resolución

1. El PR automático del frontend agrupó 18 actualizaciones y falló durante `npm ci`. Se reprodujo el error: TypeScript 7.0.2 era incompatible con `typescript-eslint` 8.67.0, que admite versiones menores que 6.1. Se corrigió la política para no agrupar ni proponer saltos mayores rutinarios; el PR incompatible debe cerrarse sin fusionar.
2. La captura de Actions también mostraba avisos por acciones basadas en Node.js 20. La rama `main` ya contenía las generaciones vigentes de `checkout`, `upload-artifact` y las acciones Docker; los avisos procedían de una rama Dependabot atrasada dos commits.

3. El primer intento de proteger el repositorio privado devolvió HTTP 403 porque el plan actual no incluye protección de ramas privadas. El usuario autorizó volver público el repositorio; la misma configuración se aplicó correctamente a `develop` y `main`.
4. Una prueba adicional `make lint` se detuvo porque SQL Server estaba no saludable después de un reinicio. El motor aceptaba conexiones, pero AdventureWorks seguía recuperándose cuando el inicializador intentó configurarla y dejó ausente la marca de disponibilidad. Se añadió una espera explícita a la base, se recreó el contenedor sin borrar el volumen y la prueba de reinicio pasó.

3.4. 24 de agosto de 2026 - Ambientes simultáneos y vista de entrega

Actividad	Estado	Resultado
Diagnóstico de acceso	Completado	Se comprobó que Vite respondía en 5173 y FastAPI estaba disponible en 8000; el error del navegador en 8080 se debía a que todavía no existía un servicio de entrega activo en ese puerto.
Vista Nginx paralela	Completado	El perfil Compose <code>delivery</code> añade <code>frontend-delivery</code> en 8080, comparte la API y las bases de desarrollo y conserva Vite en 5173.
Entrega completa aislada	Completado	<code>compose.release.yaml</code> usa el proyecto <code>bi-ia-prototype-release</code> , API 18000, PostgreSQL 55433, SQL Server 51434 y volúmenes independientes; no reemplaza desarrollo.
Automatización	Completado	<code>make delivery-preview-up</code> inicia la vista ligera. <code>make release-up</code> exige <code>.env.release</code> y levanta las imágenes GHCR; <code>make release-down</code> detiene sólo ese ambiente.
CI/CD	Completado	CI valida desarrollo, perfil de entrega y Compose GHCR mediante contenedores efímeros. CD publica desde <code>main</code> o etiquetas <code>v*</code> ; el despliegue selecciona una etiqueta sin compartir volúmenes.
Guías	Completado	README, arquitectura, réplica, flujo Git y manual técnico distinguen ramas, ambientes y consumo de recursos.
Verificación local	Completado	Cinco servicios quedaron saludables; Vite respondió en 5173, Nginx en 8080 y el proxy de entrega devolvió salud de FastAPI.
Nube académica	Pendiente	Se recomienda Azure for Students con una VM Linux x64 de 2 vCPU y 8 GiB, OIDC, aprobación del ambiente y apagado automático. Falta activar la suscripción antes de crear recursos.

Actividad	Estado	Resultado
Visibilidad GHCR	Completado	Con autorización expresa se cambió a pública la visibilidad de <code>prototipo-bi-ia-frontend</code> , <code>prototipo-bi-ia-backend</code> , <code>prototipo-bi-ia-postgres</code> , <code>prototipo-bi-ia-sqlserver</code> y <code>prototipo-bi-ia-docs</code> . GitHub confirmó el estado público de cada paquete.

3.4.1. Decisión operativa

La vista Nginx es la opción recomendada para revisar rápidamente el frontend compilado porque no duplica SQL Server. La entrega GHCR completa se reserva para validar aislamiento, imágenes y restauración reproducible; puede convivir con desarrollo si el computador dispone de memoria suficiente. La vista ligera y la entrega completa comparten por defecto el puerto anfitrión 8080, por lo que se usa sólo una de ellas a la vez o se cambia su variable de puerto.

3.4.2. Incidente de verificación documental

La primera regeneración mostró avisos porque TeX intentó crear su caché de fuentes en una ruta sin escritura. La compilación usó su alternativa temporal y `make verify` terminó correctamente, pero se fijaron `HOME`, `TEXMFVAR` y `VARTEXFONTS` a rutas temporales dentro del contenedor para eliminar la dependencia implícita y mantener iguales los comandos locales y de CI.

La primera CI del commit `7f39517` falló únicamente en `Validate Compose: .env.release.example` existía localmente, pero una regla amplia de `.gitignore` impedía versionarlo. Se añadió una excepción explícita, se incorporó el archivo de ejemplo sin credenciales reales y se repitió la verificación. Este hallazgo confirma el valor de probar desde un clon limpio además del entorno local.

3.5. 24 de agosto de 2026 - Publicación abierta de imágenes GHCR

Actividad	Estado	Resultado
Autorización	Completado	El usuario autorizó publicar las cinco imágenes porque el repositorio es académico y las colaboradoras deben poder reproducir el entorno. No se publicaron secretos, volúmenes ni respaldos privados.
Paquetes públicos	Completado	Las cinco configuraciones de GitHub Packages indican que el paquete está público; la vista del perfil muestra los cinco contenedores sin la etiqueta Private .
Acceso del equipo	Completado	LeslyVelasquez y EmyRoblesBerry conservan permiso de escritura heredado del repositorio. La lectura de los paquetes públicos no depende de ser colaborador.
Pull request 12	Pendiente	Los ocho controles de CI están aprobados y la fusión automática está activa. Falta una aprobación de cualquiera de las dos colaboradoras para integrar los tres commits de <code>chore/git-flow</code> en <code>develop</code> .
Réplica externa	Pendiente	Falta ejecutar <code>make release-up</code> desde un clon limpio en un segundo computador sin sesión de GHCR y conservar como evidencia las sondas de salud y los digest descargados.

3.5.1. Incidentes y resolución

1. La primera prueba anónima devolvió HTTP 401 porque la visibilidad del repositorio y la de GitHub Packages son controles independientes. Se corrigió manualmente la visibilidad de cada paquete desde la cuenta propietaria.
2. La ampliación de permisos de GitHub CLI fue autorizada mediante el flujo de dispositivo, pero el entorno protegido no pudo escribir temporalmente su archivo local de autenticación. Se completó la operación desde la sesión web autenticada, sin registrar contraseñas, códigos ni tokens en el repositorio.
3. El mismo bloqueo de escritura impidió actualizar la bitácora durante esa intervención. No hubo pérdida ni corrupción: el árbol Git permaneció limpio y la versión 0.1.7 registró la resolución en una sesión posterior.

3.6. 24 de agosto de 2026 - Operación documentada de contenedores

Actividad	Estado	Resultado
Manual técnico	Completado	Se elevó a la versión 1.4.0 y se añadió una sección operativa completa para los tres ambientes Docker.

Actividad	Estado	Resultado
Desarrollo	Completado	Se documentaron primera ejecución, construcción, pausa con stop , reanudación con start y retiro con make down .
Vista Nginx	Completado	Se documentaron inicio, consulta, pausa, reanudación y retiro exclusivo de frontend-delivery , sin afectar los cuatro servicios de desarrollo.
Entrega GHCR	Completado	Se documentaron release-up , pausa, reanudación, consulta y release-down con .env.release y Compose aislado.
Persistencia	Completado	El manual diferencia stop , start , down y down-volumes ; la eliminación de volúmenes queda marcada como destructiva.
Verificación	Completado	Los PDF del manual y la bitácora se regeneraron e inspeccionaron; make verify finalizó correctamente dentro de Docker.

3.7. 1 de septiembre de 2026 - Cierre documental y adopción de SDD

Actividad	Estado	Resultado
Auditoría documental	Completado	Se revisaron README, arquitectura, réplica, flujo Git, manual técnico, informe y bitácora. Se corrigió el lenguaje heredado que todavía describía repositorio o imágenes como privados; el estado vigente es repositorio público, paquetes GHCR públicos y escritura limitada a colaboradoras autorizadas.
Manual técnico	Completado	Se actualizó a versión 1.5.0 con explicación paso a paso de CI, CD, DevOps, lectura de GitHub Actions, límites actuales del despliegue y responsabilidades del equipo.
SDD	Completado	Se incorporó la guía <code>docs/sdd-workflow.md</code> , índice, plantilla y la especificación inicial <code>SPR-02-rbac-y-parametros.md</code> . A partir del Sprint 2 toda capacidad funcional debe tener una especificación versionada antes del PR de implementación.
Sprint 1	Completado	El informe se actualizó para reconocer la evidencia de réplica y el cierre documental sin declarar implementado RBAC, ETL, IA ni despliegue Azure.
Entrega Git	Pendiente	Los cambios documentales deben seguir el flujo normal: rama <code>docs/*</code> , PR hacia <code>develop</code> , CI verde, una aprobación y posterior promoción <code>develop ->main</code> .

3.7.1. Decisión de gobierno

SDD se adopta como norma de trabajo, no como una promesa de implementación automática. La especificación define alcance y criterios de aceptación; Docker y CI aportan la evidencia técnica; la bitácora registra el resultado; y el pull request conserva la aprobación. Las modificaciones meramente editoriales pueden justificar su alcance en el PR, pero todo cambio funcional, de datos, seguridad, IA o integración externa requiere especificación previa.

3.8. 2 de septiembre de 2026 - Profundización documental y material docente anónimo

Actividad	Estado	Resultado
Estado GitHub	Completado	Se verificó que el PR #15 fue fusionado en <code>develop</code> y el PR #16 promovió el cierre del Sprint 1 a <code>main</code> .

Actividad	Estado	Resultado
Manual técnico	Completado	Se elevó a versión 1.7.0 y se incorporó una reconstrucción operativa desde cero, con explicación de variables, Compose, Dockerfiles, Makefile, scripts, Git, GitHub, CI, CD, GHCR y persistencia, además de capturas reales de Actions y los controles de promoción.
Informe Sprint 1	Completado	Se corrigió el estado de cierre y se agregó una matriz de trazabilidad entre necesidades, archivos, controles y evidencias, además del ciclo DevOps aplicado.
Manual docente	Completado	Se preparó fuera del repositorio público un documento LaTeX anónimo con un caso genérico, ruta manual completa, ruta asistida por IA y diagramas de arquitectura y CI. No contiene nombres, cuentas, rutas ni identificadores del proyecto real.
Verificación	Completado	Se compilaron los cuatro PDF, se renderizaron e inspeccionaron todas sus páginas y <code>make verify</code> validó Compose, ramas, backend, frontend y documentación dentro de Docker.
Entrega Git	Completado	La mejora se consolidó y publicó en la rama <code>docs/ampliar-manuales-sprint-1</code> ; el PR #17 se abrió hacia <code>develop</code> bajo el flujo protegido después de la verificación local.

3.8.1. Criterio de separación documental

El manual técnico y el informe del sprint pertenecen a la tesis y conservan referencias verificables al repositorio real. El manual docente es un artefacto independiente y anónimo: enseña el mismo tipo de proceso mediante nombres ficticios y marcadores, sin publicar información que permita asociarlo con las estudiantes o con el prototipo.

3.9. 4 de septiembre de 2026 - Especificación UX y configuración LLM del Sprint 2

Actividad	Estado	Resultado
Revisión SDD	Completado	La especificación <code>SPR-02-rbac-y-parametros.md</code> se amplió antes de escribir código. El alcance conserva RBAC y parámetros, e incorpora la experiencia UI/UX como requisito funcional verificable.
Adaptación responsive	Completado	Se definieron contextos de prueba para móvil desde 320 px, tableta, escritorio y pantalla amplia. Se prohíbe depender de anchos fijos o generar desplazamiento horizontal involuntario.
Navegación y flujos	Completado	Se documentaron los flujos de acceso, navegación autorizada, administración de usuarios, roles/permisos y parámetros/conexión, incluidos los estados de sesión vencida y acceso denegado.
Componentes y accesibilidad	Completado	Se acordaron cascarón de aplicación, formularios, tablas, diálogos, avisos y pantallas de estado, con foco visible, etiquetas, mensajes claros y contraste verificable.
Plantilla SDD	Completado	La plantilla y el índice de especificaciones ahora exigen declarar UI/UX responsive y accesibilidad para toda capacidad futura que exponga interfaz.
Configuración LLM	Completado	Se aprobó una única configuración activa entre Gemini Cloud, Qwen Cloud u Ollama local. Sus referencias de secreto son <code>GEMINI_API_KEY</code> , <code>DASHSCOPE_API_KEY</code> y <code>none</code> , respectivamente; se conservan fuera de PostgreSQL y de la interfaz. Una prueba real y acotada confirmará conectividad sin enviar datos del negocio.
ADR de proveedores	Completado	El ADR 0002 fija adaptadores internos de FastAPI, un catálogo cerrado de tres proveedores y <code>qwen3:4b</code> como modelo local recomendado. Añadir otro proveedor requerirá actualizar decisión, especificación, controles de secretos y pruebas.
Implementación	Pendiente	No se implementaron módulos ni pantallas en esta actividad. La especificación debe ser revisada y aprobada antes de crear la rama de implementación <code>feature/rbac-y-parametros</code> .

3.9.1. Decisión de diseño

La plataforma usará un cascarón React reusable y menús derivados de permisos ya validados por FastAPI. La interfaz ofrece orientación y prevención de errores, pero no reemplaza las decisiones de autorización del backend. Se privilegiarán componentes nativos y CSS mantenible; una biblioteca visual externa sólo se evaluará con justificación explícita. La configuración del LLM será administrable con

un único proveedor activo: Gemini Cloud, Qwen Cloud u Ollama local. Su conexión será comprobable mediante una prueba real, limitada y sin datos del negocio. Las solicitudes generativas sobre BI sólo podrán ser realizadas posteriormente por el módulo `copilot`, usando una configuración activa validada y sin exponer credenciales.

3.10. 6 de septiembre de 2026 - Implementación inicial del Sprint 2

Actividad	Estado	Resultado
Migración Sprint 2	Completado	Se incorporó la revisión Alembic 20260906_01, que crea en el esquema app las tablas de RBAC, auditoría, parámetros y configuración LLM no secreta. El servicio migrate la ejecuta antes de FastAPI.
Seguridad	Completado	Se implementaron hash Argon2, JWT de vida limitada, cuenta administradora inicial por variables de entorno, permisos estables, menús autorizados y auditoría de acceso y cambios administrativos.
Parámetros y fuente externa	Completado	Se implementaron parámetros no secretos, prueba lectora de AdventureWorks mediante la cuenta configurada y adaptadores acotados para Gemini, Qwen Cloud y Ollama local. Las claves siguen exclusivamente en variables de entorno.
Interfaz base	Completado	React incorpora acceso, cierre de sesión, navegación derivada de permisos, avisos y vistas de consulta para los recursos del sprint. El cascarón usa CSS responsive desde 320 px.
Verificación integrada	Completado	En Docker se comprobó la migración, la creación del administrador, el inicio de sesión JWT, la sesión con ocho menús autorizados y las consultas protegidas de usuarios, roles, permisos, menús, parámetros, LLM y auditoría.
Pendiente de cierre	Pendiente	Faltan formularios administrativos completos en React, pruebas negativas de autorización y de adaptadores, evidencia visual responsive, make verify final y el PR hacia develop .
Alcance CRUD	Completado	La especificación se precisó para exigir CRUD completo de usuarios, roles, permisos, menús, parámetros y configuración LLM, con desactivación segura, autorización backend y auditoría.

4. Matriz de verificación de la fase

Control	Estado	Evidencia esperada
Compose autocontenido válido	Completado	<code>docker compose config -quiet</code>
Compose de imágenes válido	Completado	<code>docker compose -f compose.release.yaml config -quiet</code>
Backend calidad y pruebas	Completado	Ruff, Mypy y 2 pruebas Pytest dentro de Docker
Frontend calidad y pruebas	Completado	ESLint, 1 prueba Vitest y build de Vite dentro de Docker
PostgreSQL disponible	Completado	Esquemas <code>app</code> y <code>mart</code> ; endpoint <code>/health/ready</code> en ok
AdventureWorks restaurada	Completado	71 tablas; <code>bi_reader</code> : lectura 1 e inserción 0
Interfaz disponible	Completado	Vite saludable en 5173, Nginx opcional en 8080 y API accesible mediante proxy
PDF legible	Completado	Todas las páginas renderizadas e inspeccionadas sin cortes ni solapamientos
Git local	Completado	<code>develop</code> para integración, <code>main</code> para entregas y ramas enfocadas por pull request
GitHub/GHCR	Completado	Repositorio público, dos colaboradoras con escritura, ramas protegidas y cinco imágenes públicas en GHCR; la réplica desde una VM Ubuntu se realizó con Docker y VS Code.
SDD y trazabilidad	Completado	Guía SDD, plantilla y especificación inicial del Sprint 2 versionadas; toda capacidad funcional futura debe enlazarse a su especificación.

5. Próximos pasos

1. Publicar el cierre documental mediante PR hacia `develop`; una vez aprobado, promoverlo hacia `main` como parte de la entrega estable.
2. Revisar individualmente los pull requests de Dependabot que queden abiertos en `develop`; no fusionar ninguno sin CI aprobada.
3. Congelar una etiqueta de ambiente inicial y registrar los digest de las imágenes.
4. Aprobar la especificación `SPR-02-rbac-y-parametros.md` antes de iniciar código de seguridad o parámetros.
5. Iniciar la siguiente iteración con migraciones del RBAC y parámetros, sin adelantar módulos BI, IA o ETL.

6. Historial del documento

Versión	Fecha	Cambio
0.1.0	24-08-2026	Apertura de la bitácora y registro de la fase de ambiente, Docker, Git y DevOps.
0.1.1	24-08-2026	Prueba integral desde volúmenes vacíos, corrección del usuario lector e inicialización del repositorio Git.
0.1.2	24-08-2026	Ejecución desde la ruta definitiva y validación de la sesión GitHub de LFXAS.
0.1.3	24-08-2026	Repositorio privado, primera CI/CD exitosa, imágenes GHCR y mantenimiento de Actions.
0.1.4	24-08-2026	Informe formal del Sprint 1, manual técnico, capturas e integración de todos los PDF en CI.
0.1.5	24-08-2026	Colaboradores, estrategia <code>develop/main</code> , protección, control de ramas, Dependabot y reinicio seguro de SQL Server.
0.1.6	24-08-2026	Vista Nginx paralela, entrega GHCR aislada y documentación de ambientes simultáneos con CI/CD.
0.1.7	24-08-2026	Cinco paquetes GHCR públicos, trazabilidad del incidente de autorización y réplica externa pendiente.
0.1.8	24-08-2026	Manual técnico 1.4.0 con operación completa de desarrollo, vista Nginx y entrega GHCR.
0.1.9	01-09-2026	Cierre documental del Sprint 1: manual CI/CD/DevOps ampliado, corrección de visibilidad pública y adopción de SDD con plantilla y especificación del Sprint 2.
0.2.0	02-09-2026	Profundización del manual técnico y del informe del Sprint 1; separación de un manual docente anónimo en LaTeX y registro de los PR de cierre.
0.2.1	04-09-2026	Ampliación SDD del Sprint 2: UI/UX responsive, flujos, navegación, componentes, accesibilidad y criterios verificables previos a la implementación de RBAC.
0.2.2	04-09-2026	Se incorporó la configuración segura y parametrizable de un proveedor LLM para el prototipo, sin adelantar la generación de propuestas BI.
0.2.3	04-09-2026	Se aprobó el catálogo inicial Gemini Cloud, Qwen Cloud y Ollama local; el ADR 0002 define secreto por referencia, adaptadores y <code>qwen3:4b</code> como opción local.
0.2.4	06-09-2026	Implementación inicial del Sprint 2: migración RBAC/parámetros, JWT, auditoría, adaptadores LLM, cascarón React responsive y verificación integrada en Docker.